

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

3875 · Construct Uniform Parity Array I

LeetCode · Weekly Contest 494

Easy

Math

Parity / Bit Tricks

1. What Type of Question Is This?

This is a **constructive feasibility problem built on number parity**. There's no graph, no DP table, and no search over large state spaces — the entire problem hinges on one arithmetic fact: **the parity of $a - b$ depends only on the parities of a and b** (same parity $\rightarrow$ even difference, different parity $\rightarrow$ odd difference). Whenever a problem lets you pick between "use the value as-is" or "use a difference/sum of two values" and then asks about odd/even, it's almost always a parity-counting problem in disguise — think in terms of **how many odd numbers and how many even numbers** exist, not the actual values.

2. What Is Given?

- An array **nums1** of n **distinct** integers.
- You must build another array **nums2** of the same length n where every element is the same parity — **either all odd or all even** (you choose which, once, for the whole array).
- For every index i you must pick **exactly one** of two construction rules:

$\text{nums2}[i] = \text{nums1}[i]$	Copy the value straight across, unchanged
$\text{nums2}[i] = \text{nums1}[i] \oplus \text{nums1}[j]$	Subtract some other element (any index $j \neq i$) instead

3. What Do We Need to Find?

Return **true** if some valid choice of rule (per index) lets every element of **nums2** end up the same parity, and **false** otherwise. Note the function only needs to return a boolean — it does not need to output **nums2** itself.

Function Signature

■ C++

```
class Solution {
public:
    bool uniformArray(vector<int>& nums1) {

    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input:  nums1 = [2,3]
Output: true
```

nums1 has one even number (2) and one odd number (3) — mixed parity. Target "all odd": for index 0, 2 is even (doesn't match), so subtract instead: $\text{nums2}[0] = \text{nums1}[0] - \text{nums1}[1] = 2 - 3 = -1$ (odd). For index 1, 3 is already odd, so copy directly: $\text{nums2}[1] = 3$. Result $\text{nums2} = [-1, 3]$, both odd → **true**.

Example 2

```
Input:  nums1 = [4,6]
Output: true
```

nums1 is already uniform — both 4 and 6 are even. Target "all even": copy every element directly, $\text{nums2} = [4, 6]$, both even → **true**. No subtraction needed at all here.

Constraints

- $1 \leq n == \text{nums1.length} \leq 100$
- $1 \leq \text{nums1}[i] \leq 100$
- nums1 consists of distinct integers.

A Note Worth Highlighting

Try a few inputs by hand and you'll notice something: **you can never actually make this fail**. Every valid nums1 under these constraints has a working construction. The interesting part of this "Easy" problem isn't the code — it's proving that fact. The three approaches below build up to exactly that proof.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — There are only two possible targets.** "All odd" or "all even" — that's it. So the whole problem reduces to: *is target = all-odd achievable, or is target = all-even achievable?* Check both, return true if either works.
- **Step 2 — Learn the one arithmetic fact that matters.** For any integers a, b : $\text{parity}(a - b) = \text{parity}(a) \text{ XOR } \text{parity}(b)$. In plain words: same-parity minus same-parity is **even**; different-parity minus different-parity is **odd**. The actual magnitudes of the numbers never matter, only whether each one is odd or even.
- **Step 3 — Translate the per-index choice into a parity condition.** For a fixed target parity P and index i : either $\text{nums1}[i]$ already has parity P (use it directly), or you need some other index j with $\text{parity}(\text{nums1}[j]) = \text{parity}(\text{nums1}[i]) \text{ XOR } P$, using rule 2.
- **Step 4 — Stop thinking per-index, think in counts.** You don't need to know *which* j works — you only need to know whether a j with the required parity *exists anywhere else* in the array. That only depends on how many odd numbers and how many even numbers are in nums1 — two counters, not a search.
- **Step 5 — Push the counting argument all the way through** and you land on a clean closed-form: target-even works unless there is *exactly one* odd number; target-odd works whenever there is *at least one* odd number. Those two failure conditions ($\text{oddCount} == 1$ and $\text{oddCount} == 0$) can never both happen at once — so **at least one target is always achievable**. The answer is provably always true.

Which Algorithm Is Best?

The **optimal** approach isn't really an algorithm in the traditional sense — it's a **mathematical proof** that collapses the problem to $O(1)$: if nums1 is already uniform parity, copy it directly; otherwise (mixed parity present), subtract an opposite-parity partner from every element, which unconditionally produces an all-odd array. Either way the answer is true, every time, for every valid input. The brute-force and "better" tiers below exist to show *how* you'd discover that fact if you didn't already see it — going from "search for a witness" → "count parities and check a formula" → "realize the formula can never fail".

Approach 1 — Brute Force (Search Every Pair for a Witness)

Idea: For each of the two possible targets (all-even, all-odd), check every index independently: does the direct value already match, or does *some* other index provide a valid subtraction partner? This is the most literal translation of the problem statement into code — loop over every (i, j) pair and look for proof that index i can be satisfied.

Steps

- Try target parity = even (0) first.
- For each index i, check if `nums1[i]` itself is even. If so, this index is fine — move on.
- Otherwise, loop over every other index j and check if `nums1[i] - nums1[j]` is even. If any j works, this index is fine.
- If no j works for some index, target = even fails entirely — stop early.
- Repeat the whole process for target parity = odd (1).
- Return true if either target parity fully succeeded for all indices.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool feasible(vector<int>& nums1, int target) {
        int n = nums1.size();
        for (int i = 0; i < n; i++) {
            bool found = (nums1[i] % 2 == target);
            if (!found) {
                for (int j = 0; j < n; j++) {
                    if (j == i) continue;
                    int diff = nums1[i] - nums1[j];
                    int p = ((diff % 2) + 2) % 2; // safe parity, handles negatives
                    if (p == target) { found = true; break; }
                }
            }
            if (!found) return false; // this index has no valid choice
        }
        return true;
    }

    bool uniformArray(vector<int>& nums1) {
        return feasible(nums1, 0) || feasible(nums1, 1); // try all-even, then all-odd
    }
};

int main() {
    vector<int> nums1 = {2, 3};
    Solution sol;
    cout << (sol.uniformArray(nums1) ? "true" : "false") << endl; // Expected output: true
    return 0;
}
```

■ Python

```
class Solution:
    def uniformArray(self, nums1):
        return self.feasible(nums1, 0) or self.feasible(nums1, 1)
```

```

def feasible(self, nums1, target):
    n = len(nums1)
    for i in range(n):
        found = (nums1[i] % 2 == target)
        if not found:
            for j in range(n):
                if j == i:
                    continue
                if (nums1[i] - nums1[j]) % 2 == target:
                    found = True
                    break
            if not found:
                return False          # this index has no valid choice
    return True

if __name__ == "__main__":
    nums1 = [2, 3]
    print(Solution().uniformArray(nums1))    # Expected output: True

```

■ JavaScript

```

class Solution {
    uniformArray(nums1) {
        return this.feasible(nums1, 0) || this.feasible(nums1, 1);
    }

    feasible(nums1, target) {
        const n = nums1.length;
        for (let i = 0; i < n; i++) {
            let found = (((nums1[i] % 2) + 2) % 2) === target;
            if (!found) {
                for (let j = 0; j < n; j++) {
                    if (j === i) continue;
                    const diff = nums1[i] - nums1[j];
                    const p = ((diff % 2) + 2) % 2;
                    if (p === target) { found = true; break; }
                }
            }
            if (!found) return false;      // this index has no valid choice
        }
        return true;
    }
}

const nums1 = [2, 3];
console.log(new Solution().uniformArray(nums1)); // Expected output: true

```

Complexity: For each of the 2 targets, checking every index against every other index is $O(n) \times O(n) = O(n^2)$. With 2 targets total, that's $O(n^2)$ overall — fine given $n \leq 100$, but it re-derives the same yes/no answer for every single pair instead of using the fact that only the *parity* of each number matters.

Approach 2 — Better (Count Parities, Then Use a Closed-Form Rule)

Idea: Drop the inner search entirely. All that ever mattered was *how many* odd numbers exist — not which ones, and not the even numbers' values either. Count the odd numbers once, then answer both targets with a simple rule derived from that single count.

Steps

- Make one pass over `nums1` and count how many elements are odd (`oddCount`).
- **Target = all even** is achievable unless exactly one number is odd: if `oddCount == 0` the array is already all even; if `oddCount ≥ 2`, every odd index can borrow a *different* odd index to subtract with and become even; if `oddCount == 1`, that single odd number has no odd partner left, so it's stuck.
- **Target = all odd** is achievable whenever `oddCount ≥ 1`: every even index can subtract that one odd number (odd – even and even – odd are both odd), and any already-odd index just copies directly.
- Return true if either closed-form condition holds.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool uniformArray(vector<int>& nums1) {
        int oddCount = 0;
        for (int x : nums1)
            if (x % 2 != 0) oddCount++;

        // target = all even: fails only when there's exactly ONE odd number
        // (that lone odd index has no other odd partner to subtract with)
        bool feasibleEven = (oddCount == 0) || (oddCount >= 2);

        // target = all odd: works as soon as there's at least ONE odd number
        // (every even index can subtract that odd number to become odd)
        bool feasibleOdd = (oddCount >= 1);

        return feasibleEven || feasibleOdd;
    }
};

int main() {
    vector<int> nums1 = {2, 3};
    Solution sol;
    cout << (sol.uniformArray(nums1) ? "true" : "false") << endl;    // Expected output: true
    return 0;
}
```

■ Python

```
class Solution:
    def uniformArray(self, nums1):
        oddCount = sum(1 for x in nums1 if x % 2 != 0)

        # target = all even: fails only when there's exactly ONE odd number
        feasibleEven = (oddCount == 0) or (oddCount >= 2)

        # target = all odd: works as soon as there's at least ONE odd number
        feasibleOdd = (oddCount >= 1)

        return feasibleEven or feasibleOdd
```

```
if __name__ == "__main__":
    nums1 = [2, 3]
    print(Solution().uniformArray(nums1))    # Expected output: True
```

■ JavaScript

```
class Solution {
    uniformArray(nums1) {
        const oddCount = nums1.filter(x => x % 2 !== 0).length;

        // target = all even: fails only when there's exactly ONE odd number
        const feasibleEven = oddCount === 0 || oddCount >= 2;

        // target = all odd: works as soon as there's at least ONE odd number
        const feasibleOdd = oddCount >= 1;

        return feasibleEven || feasibleOdd;
    }
}

const nums1 = [2, 3];
console.log(new Solution().uniformArray(nums1)); // Expected output: true
```

Complexity: A single $O(n)$ pass builds the count, and the two feasibility checks are $O(1)$ each — overall $O(n)$ time, $O(1)$ extra space. This is already a fully practical, interview-ready solution; no nested loops needed anymore.

Approach 3 — Optimal (The Constructive Proof — It's Always True)

Idea: Push the Approach 2 insight one step further and it stops being a search or even a counting problem — it becomes a direct construction. If `nums1` is already uniform parity, copying it is trivially valid. If it's mixed, pick **any single** element of opposite parity and subtract it from every element: odd – even and even – odd are *both* odd, so this one rule instantly makes the entire array odd, no matter how the odd/even numbers are distributed. Either branch always succeeds, so the function can even skip building `nums2` altogether and just return true.

Steps

- Check whether `nums1` is already all-odd or all-even. If so, direct-copy works — done.
- Otherwise, remember the index of any one odd number and any one even number.
- For every element: if it's odd, subtract the remembered even element (odd–even = odd); if it's even, subtract the remembered odd element (even–odd = odd).
- Every resulting value is odd by the parity-arithmetic fact from Step 2 of the workflow, so the constructed `nums2` is always valid.
- Return true — and note that since one of the two branches above always applies, the boolean answer never actually depends on the input at all.

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    bool uniformArray(vector<int>& nums1) {
        int n = nums1.size();
        vector<int> nums2(n);

        bool allOdd = all_of(nums1.begin(), nums1.end(), [](int x){ return x % 2 != 0; });
        bool allEven = all_of(nums1.begin(), nums1.end(), [](int x){ return x % 2 == 0; });

        if (allOdd || allEven) {
            nums2 = nums1; // already uniform: copy directly
        } else {
            int oddIdx = -1, evenIdx = -1;
            for (int i = 0; i < n; i++) {
                if (nums1[i] % 2) oddIdx = i; else evenIdx = i;
            }
            for (int i = 0; i < n; i++) {
                int j = (nums1[i] % 2) ? evenIdx : oddIdx; // opposite-parity partner
                nums2[i] = nums1[i] - nums1[j]; // always odd, guaranteed
            }
        }
        return true; // provably always achievable – see the note below
    }
};

int main() {
    vector<int> nums1 = {2, 3};
    Solution sol;
    cout << (sol.uniformArray(nums1) ? "true" : "false") << endl; // Expected output: true
    return 0;
}
```

■ Python

```

class Solution:
    def uniformArray(self, nums1):
        allOdd = all(x % 2 for x in nums1)
        allEven = all(x % 2 == 0 for x in nums1)

        if allOdd or allEven:
            nums2 = list(nums1)          # already uniform: copy directly
        else:
            oddIdx = next(i for i, x in enumerate(nums1) if x % 2)
            evenIdx = next(i for i, x in enumerate(nums1) if x % 2 == 0)
            nums2 = []
            for x in nums1:
                j = evenIdx if x % 2 else oddIdx  # opposite-parity partner
                nums2.append(x - nums1[j])        # always odd, guaranteed

        return True          # provably always achievable – see the note below

if __name__ == "__main__":
    nums1 = [2, 3]
    print(Solution().uniformArray(nums1))      # Expected output: True

```

■ JavaScript

```

class Solution {
    uniformArray(nums1) {
        const allOdd = nums1.every(x => x % 2 !== 0);
        const allEven = nums1.every(x => x % 2 === 0);

        let nums2;
        if (allOdd || allEven) {
            nums2 = [...nums1];          // already uniform: copy directly
        } else {
            const oddIdx = nums1.findIndex(x => x % 2 !== 0);
            const evenIdx = nums1.findIndex(x => x % 2 === 0);
            nums2 = nums1.map(x => {
                const j = (x % 2 !== 0) ? evenIdx : oddIdx; // opposite-parity partner
                return x - nums1[j];                      // always odd, guaranteed
            });
        }
        return true;          // provably always achievable – see the note below
    }
}

const nums1 = [2, 3];
console.log(new Solution().uniformArray(nums1)); // Expected output: true

```

Complexity: Building the witness array `nums2` (for illustration) takes $O(n)$ time and $O(n)$ space. But since the branch chosen always succeeds, the boolean answer itself can be computed in $O(1)$: `return true;`. This is the natural end point of the reasoning chain — brute force found a witness by search, "better" found a closed-form condition, and "optimal" shows that condition can never actually fail.

Side-by-Side Comparison

1. Brute Force	Search every (i, j) pair for a valid witness, for both targets	$O(n^2)$	$O(1)$
2. Better	Count odd numbers once, check a closed-form feasibility rule	$O(1)$	$O(1)$
3. Optimal	Constructive proof: one rule always builds a valid nums2 to build nums2, $O(1)$ for the whole array	$O(1)$	$O(1)$

Key Takeaways to Remember

- **Parity of a difference only depends on the parities of the operands:** same-parity – same-parity = even; different-parity – different-parity = odd. This one fact is the entire problem.
- When a problem only cares about odd/even (or any small fixed set of categories), **stop tracking individual values and start tracking counts per category** — it collapses an $O(n^2)$ pairwise search into an $O(n)$ counting pass.
- Push a counting argument far enough and sometimes it reveals the answer is **constant** for every valid input — always worth asking "can this ever actually fail?" before assuming a problem needs real search or DP.
- "Easy" difficulty on LeetCode often signals exactly this: the code is short, but the *reasoning* to get to the short code is the real content of the problem.
- All three tiers were validated against **Example 1** (nums1 = [2,3] → true) across C++, Python, and JavaScript.